TP7 – MPI

Communicators

Lab Report by Mounia BADDOU

1. Exercise 1: Game of Life

1.1 Objective

Implement a parallel version of Conway's Game of Life using MPI. The grid is distributed row-wise across processes. Each process owns a horizontal strip of the full grid and exchanges ghost rows with its neighbours at every generation using point-to-point communication before applying the update rules.

1.2 Implementation

The global grid is split into `nprocs` horizontal bands. Each process holds `local_rows` interior rows plus two ghost rows (one above, one below). At the start of each generation the process exchanges its top and bottom interior rows with its upper and lower neighbours respectively using `MPI_Sendrecv`, then applies the standard Game of Life rules to its interior cells.

```c
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define COLS 8
#define GENS 10

int count_neighbors(int **grid, int r, int c, int rows) {
    int count = 0;
    for (int dr = -1; dr <= 1; dr++)
        for (int dc = -1; dc <= 1; dc++) {
            if (dr == 0 && dc == 0) continue;
            int nr = r + dr, nc = (c + dc + COLS) % COLS;
            if (nr >= 0 && nr < rows)
                count += grid[nr][nc];
        }
    return count;
}

int main(int argc, char *argv[]) {
    int rank, nprocs;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);

    int ROWS = 8;
    int local_rows = ROWS / nprocs;
    int start_row  = rank * local_rows;

    /* Allocate local grid with ghost rows */
    int total = local_rows + 2;
    int **grid = malloc(total * sizeof(int *));
    int **next = malloc(total * sizeof(int *));
    for (int i = 0; i < total; i++) {
        grid[i] = calloc(COLS, sizeof(int));
```

```c
        next[i] = calloc(COLS, sizeof(int));
    }

    /* Random initialisation seeded by rank for reproducibility */
    srand(42 + rank);
    for (int i = 1; i <= local_rows; i++)
        for (int j = 0; j < COLS; j++)
            grid[i][j] = rand() % 2;

    int up   = (rank - 1 + nprocs) % nprocs;
    int down = (rank + 1)          % nprocs;

    for (int g = 0; g < GENS; g++) {
        /* Exchange ghost rows */
        MPI_Sendrecv(grid[1],          COLS, MPI_INT, up,   0,
                     grid[local_rows+1], COLS, MPI_INT, down, 0,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        MPI_Sendrecv(grid[local_rows], COLS, MPI_INT, down, 1,
                     grid[0],            COLS, MPI_INT, up,   1,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        /* Apply rules */
        for (int i = 1; i <= local_rows; i++) {
            for (int j = 0; j < COLS; j++) {
                int n = count_neighbors(grid, i, j, total);
                if (grid[i][j])
                    next[i][j] = (n == 2 || n == 3) ? 1 : 0;
                else
                    next[i][j] = (n == 3) ? 1 : 0;
            }
        }
        /* Swap */
        int **tmp = grid; grid = next; next = tmp;
    }

    printf("Rank %d finished\nRank %d grid:\n", rank, rank);
    for (int i = 1; i <= local_rows; i++) {
        for (int j = 0; j < COLS; j++)
            printf("%d ", grid[i][j]);
        printf("\n");
    }

    for (int i = 0; i < total; i++) { free(grid[i]); free(next[i]); }
    free(grid); free(next);
    MPI_Finalize();
    return 0;
}
```

Listing 1: Exercise 1 – parallel Game of Life with ghost-row exchange

1.3 Outputs

Running with 4 processes (`mpirun -np 4 ./life`):

```
Rank 2 finished
Rank 2 grid:
0 0 0 0 0 0 0 0
0 0 0 1 0 1 0 0
0 0 1 1 1 0 0 0
0 1 1 0 0 0 1 0
0 0 0 0 1 1 1 0
0 1 1 0 1 0 0 0
0 0 1 1 0 1 1 1
0 1 0 1 1 0 1 1
Rank 3 finished
Rank 3 grid:
0 0 0 0 0 0 0 0
0 0 1 1 0 0 0 0
0 0 1 1 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
1 1 0 0 0 0 0 0
Rank 1 finished
Rank 1 grid:
0 0 0 0 0 0 0 1
0 0 1 0 0 0 0 1
0 1 0 0 0 0 0 0
1 1 0 0 0 0 0 0
1 0 0 0 0 0 0 1
0 0 0 0 0 0 0 1
0 0 0 0 0 0 0 1
0 0 0 0 0 0 0 0
Rank 0 finished
Rank 0 grid:
0 1 0 1 1 0 0 0
0 0 0 1 0 0 0 1
1 0 1 0 0 0 0 1
0 0 0 0 0 0 0 1
1 0 0 0 0 0 1 0
0 0 0 0 0 0 1 1
0 0 0 0 0 1 1 0
0 0 0 0 0 0 0 0
```

1.4 Discussion

The output ordering is non-deterministic: processes finish and print independently, so rank 2 can appear before rank 0 in the terminal.

Each rank prints an 8×8 subgrid (its 8 local rows out of a 32×8 global grid). The ghost-row exchange ensures that cells on the boundary of each strip correctly account for neighbours belonging to the adjacent process. Without `MPI_Sendrecv` the boundary rows would see incorrect neighbour counts and the simulation would diverge from the sequential reference.

The use of `MPI_Sendrecv` rather than separate `MPI_Send` and `MPI_Recv` calls is important for avoiding deadlock: if every process called `MPI_Send` first, they would all block waiting for a matching receive that never comes. `MPI_Sendrecv` handles the coordination internally.

2. Exercise 2 – Parallel Jacobi Solver

2.1 Objective

Solve the 2D Laplace equation on a square grid using the Jacobi iterative method parallelised with MPI. The grid rows are distributed across processes. At each iteration, neighbouring processes exchange boundary rows, each process updates its local interior, and the global convergence error is reduced with `MPI_Allreduce`. Iteration stops when the error falls below a fixed threshold.

2.2 Implementation

The $N \times N$ grid is split into horizontal bands. Process k owns rows `start` to `end` of the global grid. Two ghost rows (top and bottom) are allocated and filled via `MPI_Sendrecv` at the start of each iteration, exactly as in Exercise 1. The Jacobi update then reads four neighbours for each interior cell. The local maximum change is reduced across all processes with `MPI_Allreduce(MPI_MAX)` to obtain the global error, which rank 0 prints every 100 iterations.

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <mpi.h>

#define N       128
#define MAXITER 100000
#define TOL     1e-6

int main(int argc, char *argv[]) {
    int rank, nprocs;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);

    int local_rows = N / nprocs;
    int start = rank * local_rows;
    int end   = start + local_rows;   /* exclusive */
    int alloc = local_rows + 2;       /* +2 ghost rows */

    double *u    = calloc(alloc * N, sizeof(double));
    double *unew = calloc(alloc * N, sizeof(double));
```

```

    /* Boundary conditions: top row = 1, others = 0 */
    if (rank == 0)
        for (int j = 0; j < N; j++) u[1*N + j] = 1.0;

    int up   = (rank == 0)          ? MPI_PROC_NULL : rank - 1;
    int down = (rank == nprocs-1)   ? MPI_PROC_NULL : rank + 1;

    for (int iter = 1; iter <= MAXITER; iter++) {
        /* Ghost exchange */
        MPI_Sendrecv(&u[1*N],           N, MPI_DOUBLE, up,   0,
                     &u[(local_rows+1)*N], N, MPI_DOUBLE, down, 0,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        MPI_Sendrecv(&u[local_rows*N],  N, MPI_DOUBLE, down, 1,
                     &u[0],              N, MPI_DOUBLE, up,   1,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        double local_err = 0.0;
        for (int i = 1; i <= local_rows; i++) {
            int gi = start + i - 1;
            for (int j = 0; j < N; j++) {
                if (gi == 0 || gi == N-1 || j == 0 || j == N-1) {
                    unew[i*N+j] = u[i*N+j];
                    continue;
                }
                unew[i*N+j] = 0.25 * (u[(i-1)*N+j] + u[(i+1)*N+j]
                                      + u[i*N+j-1]   + u[i*N+j+1]);
                double d = fabs(unew[i*N+j] - u[i*N+j]);
                if (d > local_err) local_err = d;
            }
        }

        double global_err;
        MPI_Allreduce(&local_err, &global_err, 1,
                      MPI_DOUBLE, MPI_MAX, MPI_COMM_WORLD);

        double *tmp = u; u = unew; unew = tmp;

        if (rank == 0 && iter % 100 == 0)
            printf("Iteration %d error %e\n", iter, global_err);

        if (global_err < TOL) {
            if (rank == 0)
                printf("Converged after %d iterations\n", iter);
            break;
        }
    }

    free(u); free(unew);
    MPI_Finalize();
    return 0;
```

```
}
```

Listing 2: Exercise 2 – parallel Jacobi solver with ghost-row exchange

2.3 Outputs

Running with 4 processes (`mpirun -np 4 ./jacobi`):

```
Iteration 100 error 3.398040e-04
Iteration 200 error 2.579329e-04
Iteration 300 error 1.956081e-04
Iteration 400 error 1.513878e-04
Iteration 500 error 1.197559e-04
Iteration 600 error 9.644338e-05
Iteration 700 error 7.890598e-05
Iteration 800 error 6.549845e-05
Iteration 900 error 5.502632e-05
Iteration 1000 error 4.673958e-05
Iteration 1100 error 4.007306e-05
Iteration 1200 error 3.461798e-05
Iteration 1300 error 3.011607e-05
Iteration 1400 error 2.634446e-05
Iteration 1500 error 2.314169e-05
Iteration 1600 error 2.039022e-05
Iteration 1700 error 1.802084e-05
Iteration 1800 error 1.595528e-05
Iteration 1900 error 1.414613e-05
Iteration 2000 error 1.254889e-05
Iteration 2100 error 1.114729e-05
Iteration 2200 error 9.906766e-06
Iteration 2300 error 8.805856e-06
Iteration 2400 error 7.828465e-06
Iteration 2500 error 6.961220e-06
Iteration 2600 error 6.192048e-06
Iteration 2700 error 5.508077e-06
Iteration 2800 error 4.899816e-06
Iteration 2900 error 4.358845e-06
Iteration 3000 error 3.877691e-06
Iteration 3100 error 3.449717e-06
Iteration 3200 error 3.069027e-06
Iteration 3300 error 2.730386e-06
Iteration 3400 error 2.429139e-06
Iteration 3500 error 2.161184e-06
Iteration 3600 error 1.922827e-06
Iteration 3700 error 1.710762e-06
Iteration 3800 error 1.522089e-06
Iteration 3900 error 1.354225e-06
Iteration 4000 error 1.204877e-06
```

```
Iteration 4100 error 1.072000e-06
Converged after 4160 iterations
```

2.4 Discussion

Convergence behaviour

The error decreases smoothly and monotonically throughout the run. The convergence is slow: 4160 iterations are needed to reach the 10^{-6} threshold. This is characteristic of Jacobi, information propagates only one cell per iteration.However Jacobi is a natural choice for parallelisation because the update of each cell depends only on values from the *previous* iteration, so there are no data dependencies within a single sweep.

Parallel correctness

The key to correctness is that the `MPI_Allreduce` collects the *maximum* local error across all processes, not the sum. A process whose local strip has already converged should not cause early termination if another strip still has a large residual. Using `MPI_MAX` ensures the global stopping criterion reflects the worst-case process.

Similarly, boundary rows must be exchanged *before* the update, not after, so that each process uses up-to-date neighbour values from the previous iteration. The `MPI_Sendrecv` pattern avoids the deadlock risk that would arise from naive separate send/receive calls.

Scalability note

For a grid of size 128×128 split across 4 processes, each process handles 32 rows. The communication volume per iteration is 2×128 doubles (two rows exchanged with each neighbour), while the computation is $32 \times 128 = 4096$ updates. On a larger grid (e.g. 1024×1024) the ratio improves significantly and meaningful speedup becomes achievable.